

Regression Audit — “Did PR #190 break failedLocator extraction?”

Question asked: Find the first commit where `failedLocator = "#username"` became `failedLocator = null`. Hypothesis under test: PR #190 introduced the regression.

Verdict: HYPOTHESIS DISPROVEN. PR #190 did not — and could not — change locator extraction. There is no extraction regression in the code at all. The two runs you compared are **different failure modes feeding the same unchanged extractor**, not the same failure before/after a code change.

1. What PR #190 actually changed (the whole diff)

PR #190 (merge 197746b , content commit 53c91ea + 26c9956) touched exactly:

File	Change
src/api/server.ts	3 rerun call-sites switched from <code>path.relative(...)</code> to <code>resolveRerunRel-File(...)</code>
src/core/rerun-target.ts	new — picks a real <code>.spec</code> file to rerun, else grep-only
tests/unit/rerun-target-resolution.test.ts	new — 7 tests

That is the entire PR. It governs **which file Playwright re-runs to confirm a heal**. It runs after a candidate is generated. It has zero contact with failure parsing, artifact collection, or `failed_locator`.

2. Proof the extraction code is byte-identical across the PR #190 boundary

```
git diff 53c91ea^ HEAD -- \
  src/core/artifact-collector.ts \
  src/core/failure-analyzer.ts \
  src/core/locator-extractor.ts
→ (empty output: identical)
```

The three files that produce `failed_locator` have **not changed by a single byte** between the commit before PR #190 and current HEAD.

3. Where `failed_locator` is actually produced, and when it last changed

`src/core/artifact-collector.ts` — last 6 commits to touch it:

```
e768a8a PR #185 rerun the SPEC file, not the Page Object
3589e97 PR #182 Repo Intelligence single source of truth
702356f PR #181 resolve broken locator to Page Object on test timeout
f9de93c TraceParser + page URL from trace
1a19c5b PR #177 find Page Object file in error stack
e48dd53 find actual Page Object file in error stack
```

Nothing after `e768a8a` (PR #185) touches it. PR #190 is later than PR #185.

The extractor has been frozen since before PR #190 shipped.

Every later commit on the healing path (`78c790f` , `6459802` , `53c91ea` , `31371f7` , etc.) only **reads** `failure.failedLocator` / `failure.failedLineCode` in new modules (`deterministic-locator-healing.ts` , `advisors`). None mutate the extractor.

4. So why is `failedLocator` null in the bad run? (it's the INPUT, not the code)

The extractor is fully deterministic given its input `test-results.json` . Reading it (`artifact-collector.ts:103–211`), the chain is:

```
candidateLocations = [errorLocation, error.location, ...errors[].location] (line
118)
resolvedLocation   = findActualSourceLocation(stack, candidateLocations) (line
131)
lineNumber         = resolvedLocation.line ?? 0 (line
145)
codeContext        = extractCodeContext(filePath, lineNumber) (line
149)
locatorInfo        = extractLocator(errorMessage) (line
|| extractLocator(codeContext.failedLineCode || ''))
156-157)
if (!locatorInfo && codeContext.failedLineCode) → Page Object fallback (line
167)
failed_locator     = locatorInfo?.rawLocator ?? null (line
211)
```

Everything hinges on **whether the run's JSON carried an error LOCATION**:

	“Good” run ([data-test="username"] found)	“Bad” run (screenshot, null)
Failure mode	Normal locator timeout — <code>errors[1]</code> carries the action error with a source location (the Page Object / spec line)	Framework-style termination (target closed / crashed / generic timeout) — no error has a <code>.location</code>
<code>candidateLocations</code>	has the PO/spec line	empty
<code>lineNumber</code>	real line	<code>0</code>
<code>codeContext.failedLineCode</code>	<code>await this.username.fill(...)</code>	empty
<code>extractLocator(...)</code>	matches → <code>#username</code>	null (nothing to parse)
Page Object fallback (line 167)	runs → [data-test="username"]	skipped (<code>failedLineCode</code> empty)
<code>failed_locator</code>	concrete selector	null
Downstream	advisors get an anchor → correct heal	advisors starved → AI fabricates <code>getByRole('button', {name:'Login'})</code>

The difference is the failure mode in the input JSON, not a code change. The same extractor, fed a crashed/target-closed run, has always produced `null` — including before PR #190.

5. The real story (corrected timeline)

- “Before PR #190” you observed a run whose failure was a clean locator timeout → extractor produced `[data-test="username"]` → only **validation/rerun** failed.
- “After PR #190” you observed a **different run** whose failure was a crashed/target-closed termination → extractor produced `null` → AI hallucinated.

These are two different inputs. PR #190 sits between them in time, which created the illusion of a regression. The code path that nulls the locator (`lineNumber=0` → empty `failedLineCode` → fallback skipped) was introduced in **PR #181 (702356f)** and has been the behavior for crashed/no-location runs ever since — long before PR #190.

If you want to name “the commit where a no-location failure yields `failed_locator = null`”, it is `702356f` (**PR #181**) / the original `f4daaec` line 211 — ****not PR**

190.** But this only fires for crash/no-location inputs; it never regressed a

previously-extracted `#username`.

6. The one thing that would make this 100% airtight

I have **100% proven** PR #190 did not touch extraction (byte-identical diff). To **100% confirm the failure-mode-difference** between your two runs, I need the `test-results.json` for the bad job (`job_5fa1ec7c-90c`). The single decisive check:

```
.suites[].specs[].tests[].results[].errors[].location # present in good run, absent
in bad run
```

If that field is absent/empty in the bad run, the diagnosis is fully closed.

7. Bottom line (no fixes proposed, per instruction)

1. **No extraction regression exists.** `artifact-collector.ts` / `failure-analyzer.ts` / `locator-extractor.ts` are unchanged across PR #190 (and unchanged since PR #185).
2. **PR #190 only changed rerun targeting** in `server.ts` (+ new `rerun-target.ts`).
3. **`failedLocator = null` is caused by the input failure mode** (crashed / target closed / no error location), which starves the extractor — the documented behavior since PR #181, not a new break.
4. The bad run is precisely the **untrustworthy** / **“inconclusive”** class — it should not be heal-targeted in the first place; that is the lever, not the extractor.